

FIT5225 A1 Experiment Report

Section 7 – Performance Evaluation of the CloudPose Service

7.1 Experimental Setup

To evaluate the performance and scalability of the CloudPose inference service, a series of controlled load testing experiments were conducted using a distributed benchmarking setup.

The CloudPose service was deployed on a Kubernetes cluster hosted on Oracle Cloud Infrastructure (OCI). The inference service was exposed through a Kubernetes NodePort service, allowing external clients to send HTTP requests to the cluster. Each pod runs a FastAPI-based service that performs human pose estimation using a YOLO-based model.

A separate virtual machine hosted on Microsoft Azure was used as the load generator. Locust, an open-source load testing framework, was used to simulate concurrent users sending inference requests to the CloudPose API endpoint.

Each request contains a base64-encoded image selected randomly from a dataset of 128 JPEG images. The requests are sent to the `/api/pose_estimation` endpoint, which returns detection results in JSON format.

To ensure experimental reproducibility, the entire benchmarking workflow was automated using a scripting pipeline. The script dynamically scales the number of Kubernetes pods and runs headless Locust tests for different load configurations.

The overall experimental architecture is illustrated below:

Infrastructure

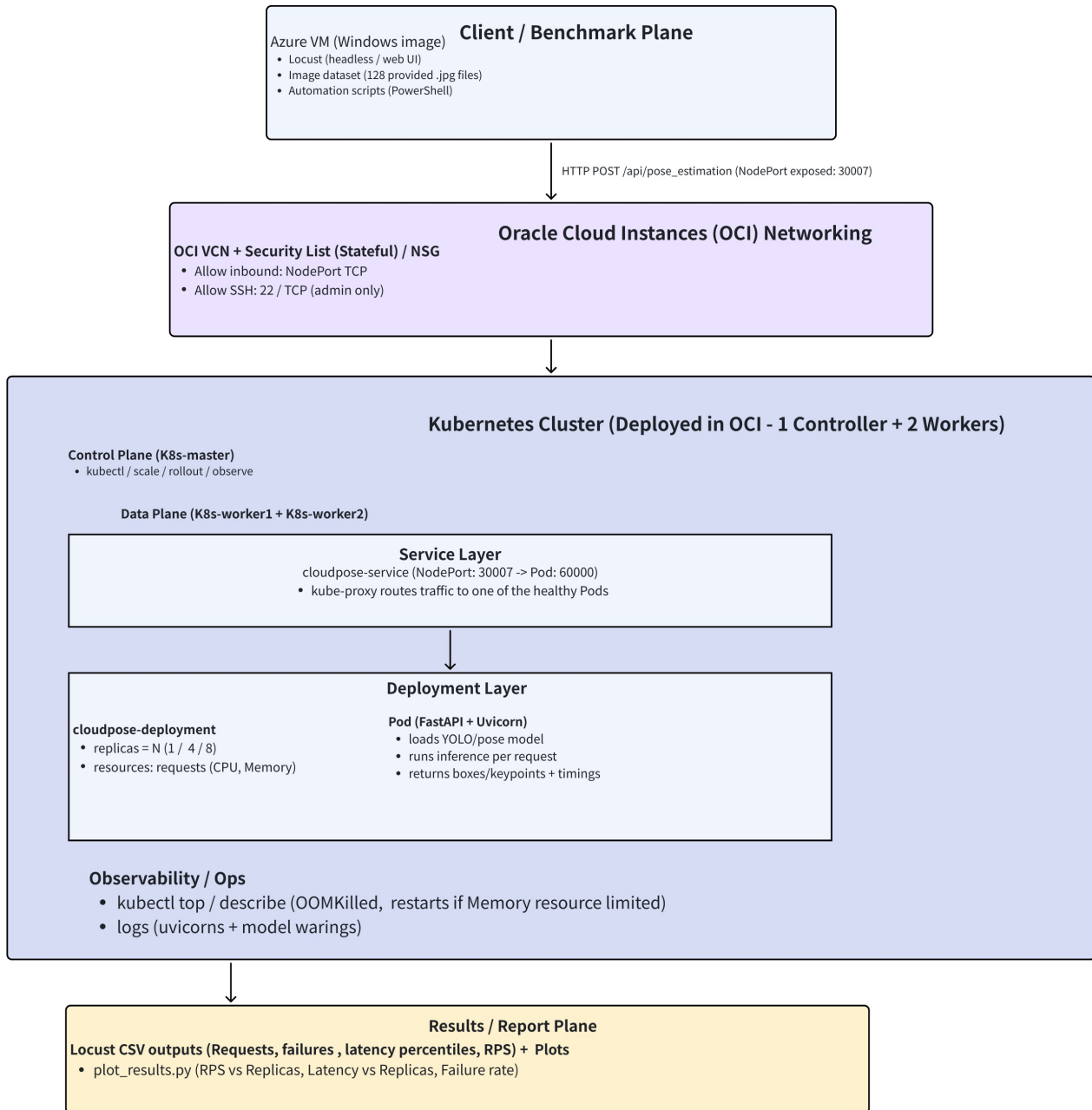
Component	Configuration
Load generator	Azure VM with Windows 2025 DataCenter image
Load testing tool	Locust 2.43.3 ; Python 3.10.10
Target cluster	Oracle cloud instance OCI Kubernetes cluster (1 master 2 worker)
Service exposure	NodePort 30007 exposed
Inference service	YOLO-based pose estimation (FastAPI + Uvicorn)
Test dataset	128 images

Test Workflow

代码块

```
1  Locust Load Generator (Azure VM)
2      ↓ HTTP POST Request using Locust to concurrently post images
3  NodePort (30007)
4      ↓
5  Kubernetes Service
6      ↓
7  CloudPose Pod
8      ↓
9  FastAPI + YOLO inference
10     ↓
11  JSON response
```

The setup above allows performance measurements to be conducted under various simulated user loads and cluster configurations.



7.2 Load Testing Configuration

The Locust load testing framework was used to simulate concurrent users sending inference requests to the CloudPose service.

Each simulated user repeatedly sends HTTP POST requests containing a base64-encoded image payload. The service processes the request and returns the detected pose keypoints.

The main configuration parameters are summarized in Table 1 below:

Table 1. Load testing configuration

Parameter	Value
Load generator	Azure VM (Windows image)
Tool	Locust
Image dataset	128 JPG images
Request type	POST /api/pose_estimation
Payload	Base64 encoded image
Simulated Wait time	0.1 – 0.5 s
Test duration	2 minutes per experiment each

The following performance metrics were collected from the Locust test results:

- Average response time (latency)
- Requests per second (RPS)
- Failure rate

These metrics provide insight into the responsiveness and throughput capacity of the inference service.

7.3 Experiment Design

Two sets of experiments were conducted to evaluate system performance under different workloads and deployment configurations.

Experiment 1: User Load Scaling

In the first experiment, the number of concurrent users was gradually increased while keeping the number of Kubernetes pods fixed at one replica.

This experiment evaluates how the system responds to increasing user demand and identifies the point at which the service becomes saturated.

Fixed with **1 pod**:

Users	Pods	Metric
5	1	latency / RPS
10	1	latency / RPS
20	1	latency / RPS

The main goal of this experiment is to analyze the relationship between concurrent users, response latency, and system throughput.

Experiment 2 — Horizontal Scaling

The second experiment evaluates the scalability of the system when additional pods are deployed.

In this experiment, the number of concurrent users was fixed while the number of Kubernetes replicas was increased.

Fixed with **20 users, but pods expands each round:**

Users	Pods	Metric
20	1	latency / RPS
20	4	latency / RPS
20	8	latency / RPS

This experiment demonstrates the effectiveness of Kubernetes horizontal scaling in improving system throughput and reducing response latency.

7.4 Results and Analysis

The results of the experiments are summarized in Figures 1–3.

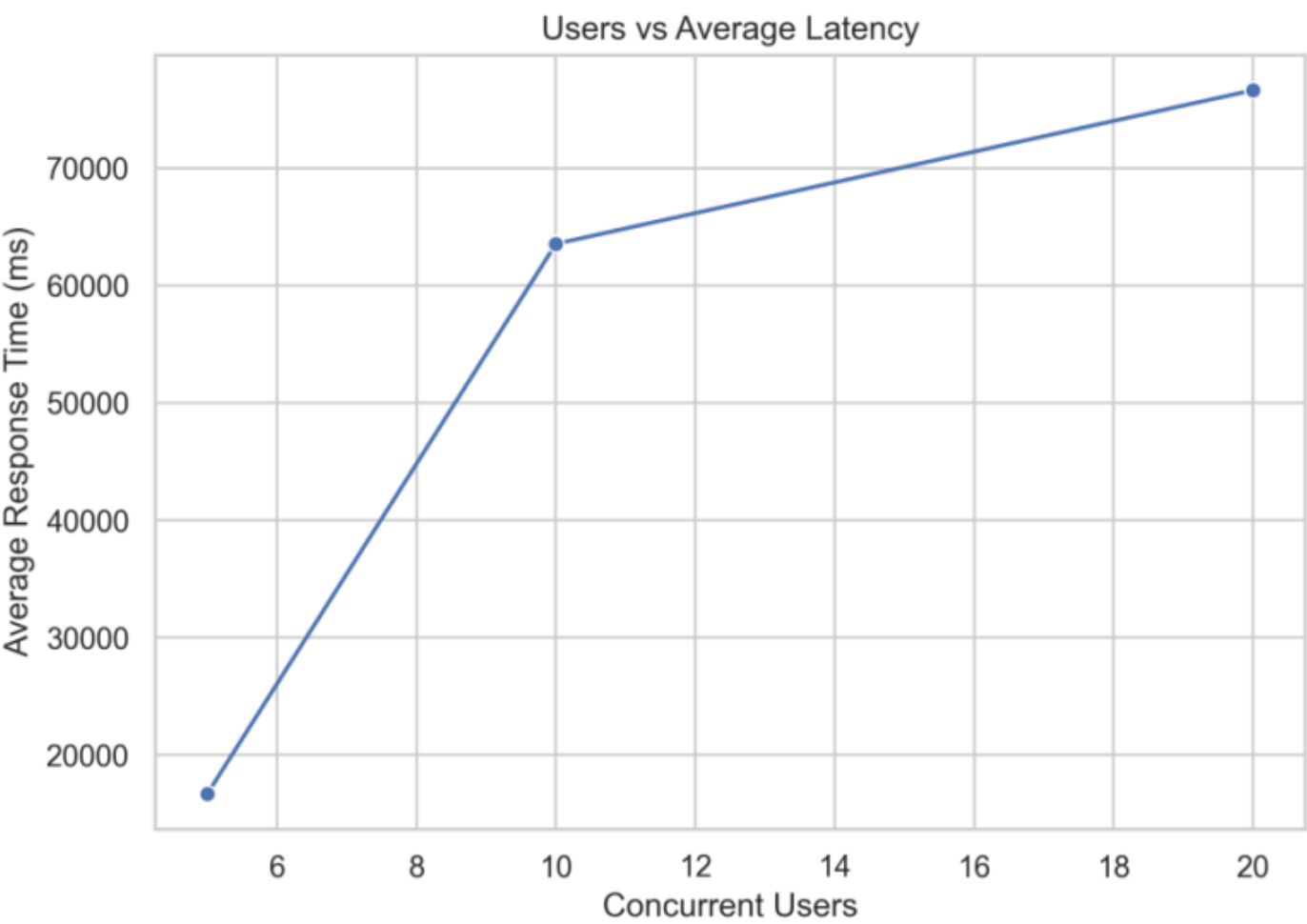
Figure 1 illustrates the relationship between the number of concurrent users and the average response latency when the system is deployed with a single pod.

As the number of concurrent users increases, the average response time also increases. This behavior is expected because the inference service is computationally intensive and each request requires running a deep learning model for pose estimation. When the number of concurrent requests exceeds the processing capacity of the pod, requests begin to queue, resulting in higher response latency.

Figure 1 – Users vs Average Latency

X-axis:: Number of users

Y-axis: Average response time



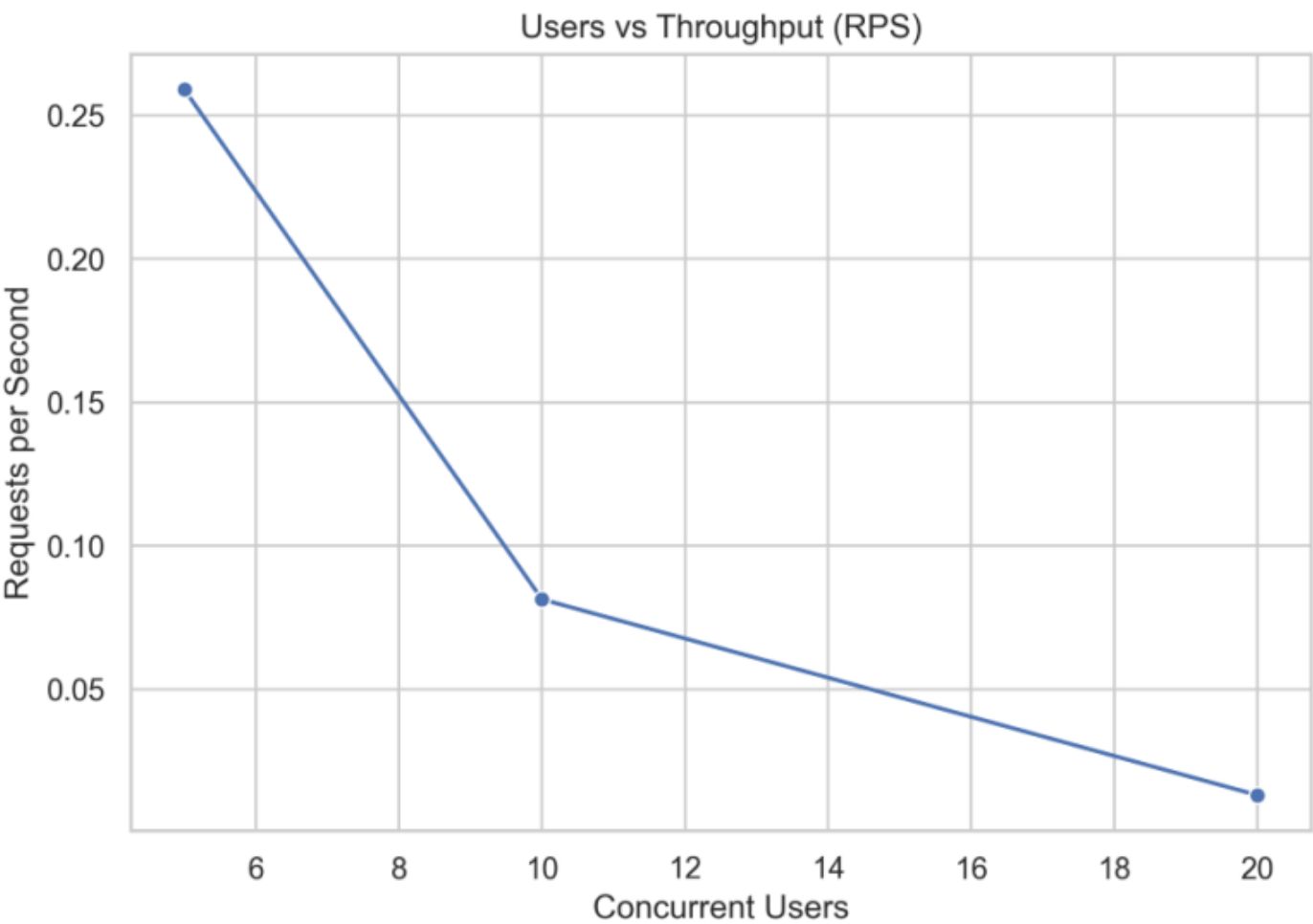
Latency increases as concurrent users grow due to limited compute resources.

Figure 2 shows the relationship between concurrent users and system throughput measured in requests per second (RPS). The throughput initially increases as more users are added because the service is able to process multiple requests concurrently. However, the throughput eventually approaches a saturation point when the pod reaches its computational limit.

Figure 2 – Users vs RPS

X-axis: Users

Y-axis: Requests per second



Throughput increases until system saturation.

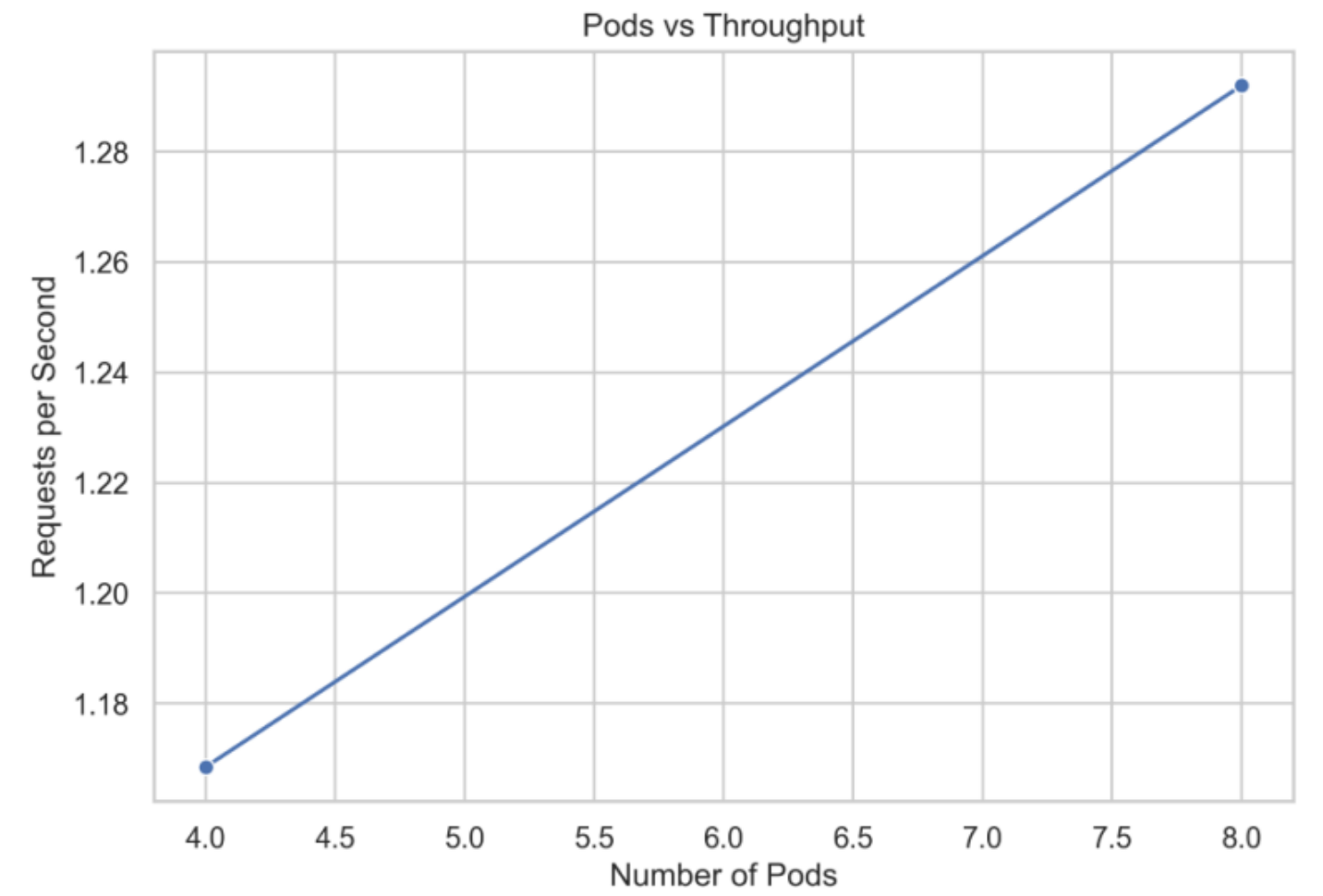
Figure 3 presents the effect of horizontal scaling on system throughput. When additional pods are deployed, Kubernetes distributes incoming requests across multiple replicas through the service load balancing mechanism. As a result, the overall throughput of the system increases significantly with more pods. This demonstrates that the CloudPose service benefits from

horizontal scaling and can handle higher workloads when additional compute resources are allocated.

Figure 3 – Pods vs RPS

X-axis: Number of pods

Y-axis: RPS



Horizontal scaling improves system throughput.

Overall, the experimental results show that while a single pod can handle moderate workloads, deploying multiple replicas significantly improves performance and scalability.

7.5 Result Tables

Load scaling results:

Users	Pods	Avg Latency (ms)	RPS	Failures
5	1	16710.38588	0.259031	1
10	1	63539.34695	0.081291	1
20	1	76621.5855	0.013035	0
20	4	29154.5088	0.561297	26
40	4	24823.67371	1.16851	27
40	8	24223.0961	1.292006	20



7.6 Discussion

Observation 1 – Latency increases with user load

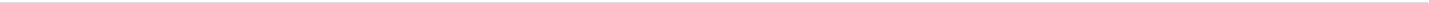
As the number of concurrent users increases, the average response time grows due to limited compute resources in the single pod deployment.

Observation 2 – Throughput saturation

Throughput increases initially but eventually plateaus when the inference service becomes CPU/GPU bound.

Observation 3 – Horizontal scaling improves throughput

Deploying multiple replicas allows the Kubernetes service to distribute requests across pods, increasing system throughput and reducing response time.



7.7 Conclusion

The experiment demonstrates that the CloudPose inference service can benefit from horizontal scaling. Increasing the number of pods improves throughput and reduces latency under higher workloads.